



TGFormer: Towards temporal graph transformer with auto-correlation mechanism

Hongjiang Chen^a, Pengfei Jiao^{a,*}, Ming Du^a, Xuan Guo^b, Zhidong Zhao^a, Di Jin^b, Xiao Liu^c

^a Hangzhou Dianzi University, School of Cyberspace, Hangzhou, 310018, China

^b Tianjin University, College of Intelligence and Computing, Tianjin, 300072, China

^c State Key Laboratory of Systems Medicine for Cancer, Shanghai Cancer Institute, Shanghai, 200127, China

ARTICLE INFO

Keywords:

Temporal graph
Graph transformer
Graph neural network
Representation learning

ABSTRACT

The growing interest in Temporal Graph Neural Networks (TGNNs) stems from their ability to model complex dynamics and deliver superior performance. However, TGNNs encounter fundamental challenges in capturing long-term dependencies and identifying periodic patterns. To address these limitations, we propose TGFormer, a novel Transformer architecture specifically designed for temporal graphs. Our model redefines temporal graph learning by establishing a trajectory framework that aligns with time series analysis principles. This approach allows TGFormer to derive node representations through systematic analysis of historical interactions, enabling granular examination of node relationships across sequential timestamps. Building upon stochastic process theory, we develop an auto-correlation mechanism that systematically uncovers periodic dependencies in node interactions. This innovation empowers TGFormer to perform dependency discovery and representation aggregation at sub-interaction levels, demonstrating superior efficiency and accuracy compared to conventional attention mechanisms. Experimental validation across six public benchmarks confirms the effectiveness of our approach, with TGFormer at most achieving 9.35% precision improvement compared to state-of-the-art approaches.

1. Introduction

Recent years have witnessed a significant increase in research focused on representation learning for temporal graphs [1]. This surge is driven by the applicability of temporal graphs in modeling dynamic systems such as social networks [2], traffic networks [3], and financial transaction systems [4]. Temporal Graph Neural Networks (TGNNs) have emerged as a promising framework by effectively capturing complex temporal dependencies, thereby enabling significant improvements in tasks such as dynamic community detection [5], traffic flow prediction [6], and real-time recommendation systems [7]. TGNN methodologies are primarily categorized into discrete-time temporal graph (DTTG) methods [8] and continuous-time temporal graph (CTTG) methods [9]. DTTG methods process graph snapshots at predetermined intervals, which may lead to information loss, whereas CTTG methods directly model event sequences to capture the finer nuances of temporal irregularities [10]. This clear distinction motivates our focus on advancing CTTG learning techniques.

Despite the successes of current CTTG methods, they encounter two critical challenges in modeling dynamic interactions. First, the popular memory-based and walk-based methods, such as TGN [11] and CAWN [12], rely on extensive historical data storage or sampling [13], which limits their ability to model long-term dependencies essential for applications like predicting user engagement trends in social networks over months or years. Second, most methods frequently fail to capture intrinsic periodic patterns that dominate many real-world systems [14,15]. These patterns, such as e-commerce platforms exhibit weekly cycles with sales peaks on weekends while streaming services experience daily viewership surges during evening hours are crucial for forecasting future behavior. Transformer-based models like DyGFormer [16] attempt to address these issues by extending historical context through attention mechanisms, as they naturally capture long-term dependencies. However, their standard attention design acts as a low-pass filter, averaging historical data and smoothing out high-frequency periodic signals [17]. This behavior, rooted in the attention mechanism's tendency to assign diffuse weights across time steps, has

* Correspondence to: Xiasha Higher Education Zone, Hangzhou, 310018, Zhejiang Province, China.

E-mail addresses: hchen@hdu.edu.cn (H. Chen), pjiao@hdu.edu.cn (P. Jiao), mdu@hdu.edu.cn (M. Du), guoxuan@tju.edu.cn (X. Guo), zhaozd@hdu.edu.cn (Z. Zhao), jindi@tju.edu.cn (D. Jin), lxc417@sjtu.edu.cn (X. Liu).

<https://doi.org/10.1016/j.patcog.2025.112053>

Received 29 September 2024; Received in revised form 5 April 2025; Accepted 23 June 2025

Available online 4 July 2025

0031-3203/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1

The qualitative comparison of the current method.

Method	Data type	Transformer	Period pattern
Autoformer [17]	Time series	✓	✓
FEDformer [22]	Time series	✓	✗
SGFormer [23]	Static graph	✓	✗
Diffformer [24]	Static graph	✓	✗
GraphMixer [25]	Temporal graph	✗	✗
DyGFormer [16]	Temporal graph	✓	✗
SimpleDyG [26]	Temporal graph	✓	✗
TGFormer	Temporal graph	✓	✓

been shown in empirical studies [18,19] to obscure sharp periodic fluctuations, such as weekend sales spikes, treating them as noise rather than predictive features. As a result, these methods yield sub-optimal predictions in scenarios where periodic patterns are central to understanding temporal dynamics.

To address these challenges, we introduce TGFormer, a novel Transformer-based architecture designed for CTTGs, which efficiently captures long-term dependencies using a Transformer framework, leveraging its ability to process historical data effectively. To handle periodic patterns, TGFormer treats temporal graph learning as a time-series analysis task and employs an auto-correlation mechanism (ACoM) instead of traditional attention mechanisms. Drawing from stochastic process theory [20], ACoM transforms time series from the time domain to the frequency domain using the Fast Fourier Transform (FFT) [21], where periodic patterns are represented as distinct frequency components, enabling the model to focus on specific frequency bands. This frequency domain approach is crucial because standard attention mechanisms, operating in the time domain, tend to smooth out high-frequency signals by averaging historical data, treating periodic fluctuations as noise, and obscuring them. By contrast, ACoM's frequency domain operation preserves these fluctuations, enhancing the capture of periodic patterns while reducing computational complexity from $O(L^2)$ to $O(L \log L)$. Our extensive experiments on six real-world datasets demonstrate TGFormer's effectiveness, achieving at most up to 9.35% improvement in precision compared to state-of-the-art methods, validating its ability to accurately model both long-term dependencies and periodic structures in temporal graphs.

The primary contributions of this work are summarized as follows:

- Treating temporal graph learning as a time-series analysis problem is a novel perspective, enabling the application of time-series techniques to capture complex temporal patterns, including periodicities, which are crucial for temporal graph representation learning.
- Based on this perspective, we design the Series Transformer layer, a novel component that effectively captures long-term dependencies in temporal graphs by modeling historical data using Transformer-based architecture.
- We introduce the auto-correlation mechanism, a novel method that complements the Series Transformer layer by capturing periodic patterns through sequence auto-correlation, enhancing the model's ability to handle time series data with reduced computational complexity ($O(L \log L)$).
- We validate TGFormer's performance through comprehensive experiments on six real-world temporal graph datasets, showing that it consistently outperforms state-of-the-art methods, demonstrating its effectiveness in capturing both long-term dependencies and periodic structures.

2. Related work

2.1. Temporal graph neural networks

In recent years, research on TGNs has garnered increasing attention. These models typically conceptualize temporal graph data as

event streams, enabling the direct learning of node representations from continuously evolving interactions. Specifically, existing CTTG models predominantly employ recurrent neural networks (RNNs) or attention mechanisms as their sequence modeling components. For instance, JODIE [27] utilizes an RNN to integrate static and dynamic embeddings for node representation learning, while TGAT [28] extends the graph attention mechanism to capture time-aware representations. Building upon these foundations, several approaches incorporate additional techniques to enhance the modeling of continuous temporal dynamics. For example, TGN [11], a variant of TGAT, introduces a memory module to effectively track node-level feature evolution. GSNOP [29] employs a sequential ordinary differential equation (ODE) aggregator, leveraging sequential dependencies to learn the derivative of neural processes, thereby improving distribution estimation from limited historical data. Simultaneously, CAWN [12] extracts temporal network motifs using set-based anonymous random walks, effectively capturing dynamic structural patterns. NeurTW [30] further enhances spatial and temporal interdependencies during anonymous random walks, integrating continuous evolution with transient activation processes to model the underlying spatiotemporal dynamics of base-order coding. To address long-term temporal dependencies in temporal graph interactions, DyGFormer [16] adopts a Transformer-based architecture, incorporating a neighbor co-occurrence encoding scheme alongside a patching technique to refine representation learning.

In a nutshell, most existing TGNs struggle to manage nodes with longer interactions and effectively capture periodic temporal patterns due to the prohibitive computational costs of complex modules and optimization challenges such as vanishing or exploding gradients. In this paper, we propose a novel transformer tailored for temporal graphs to show the necessity of long-term temporal dependencies and periodic dependencies, which is achieved by the Series Transformer layer.

2.2. Transformers on graphs

Transformer [31] is an innovative model that employs the SAM to handle sequential data, which has demonstrated significant success across diverse domains. Recent advances in graph learning have begun to assimilate Transformer models in diverse formulations. For instance, NAGphormer [32] employs sampling-based node selection, while Diffformer [24] proposes continuous-time diffusion for dynamic graphs. Expformer [33] enhances structural awareness through expander graph augmentation and virtual nodes, outperforming traditional sparse Transformers. NodeFormer [34] leverages Gumbel-Softmax kernels for all-pair message passing, and SGFormer [23] shows that just using a one-layer transformer network can sometimes improve the results of GCN-based networks and the low memory footprint can help scale to large networks. SimpleDyG [26] re-conceptualizes temporal graphs as a sequence added to the attention mechanism.

Although some studies have explored the potential of transformers for temporal graphs, they have only enhanced the inputs to the transformer and have not proposed a transformer that can accommodate characteristics unique to temporal graphs, such as periodicity. Addressing this, our work architects an auto-correlation methodology to unearth the periodic dependencies of node interactions and undertake dependency discovery and representation aggregation at the sub-interaction level.

Remark. For qualitative comparison, we summarize recent major temporal graph learning and transformer learning methods in Table 1. The comparison encompasses three aspects: applicability to temporal Graph, adoption of Transformer learning manner, and consideration of period temporal pattern. To the best of our knowledge, our approach is the only one that simultaneously satisfies all three aspects.

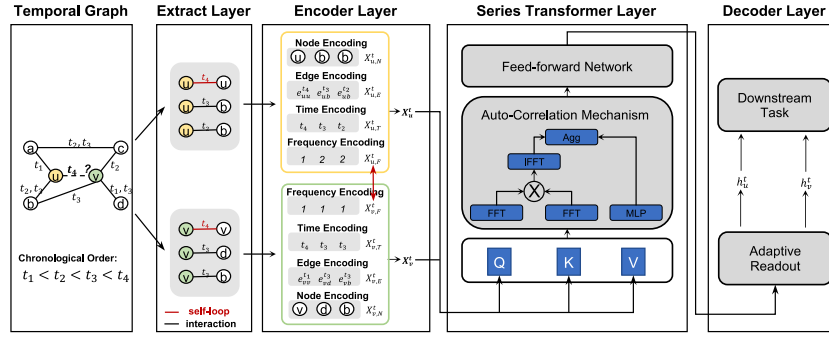


Fig. 1. The overview of TGFormer begins with the extract layer, which employs a direct interaction extractor. Subsequently, the encoder layer generates temporal-aware and structure-aware sequence node representations. These representations are then used to form the time series data placed into the query (Q), key (K), and value (V) matrices in the series transformer layer. The series transformer layer leverages the Transformer's ability to capture long-term temporal dependencies and the auto-correlation mechanism to uncover inner periodic temporal patterns in the series data. Finally, the decoder layer utilizes adaptive readout for various downstream tasks.

Table 2
Symbols and their definitions.

Notation	Description
$\mathcal{G}$	A temporal graph
$\mathcal{V}, \mathcal{E}$	The node set and link set of $\mathcal{G}$
S_u^t	The historical interactions samples of node u before time t with itself
$X_{u,N}^t$	Node u 's representation of node features at time t
$X_{u,E}^t$	Node u 's representation of link features at time t
$X_{u,T}^t$	Node u 's representation of temporal information at time t
$X_{u,F}^t$	Node u 's representation of node interactions frequency at time t
F_u^t	The number of times a node appears in S_u^t and S_v^t , respectively
X_u^t	The combine node u representation at time t
h_u^t	The node u representation at time t
d_N, d_E, d_T, d_F, d	The dimension of $X_{u,N}^t, X_{u,E}^t, X_{u,T}^t, X_{u,F}^t$
$\{X_u^t\}_{i=1}^L$	A time series
δ	The time delay
k	The select k series
L	The node historical interactions extract length

3. Preliminaries

3.1. Temporal graph

A temporal graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of nodes and $\mathcal{E}$ denotes the set of node interactions, each associated with a timestamp. For any two nodes u and v , there may exist a sequence of interactions occurring at different timestamps, formally expressed as $\mathcal{E}_{u,v} = \{(u, v, t_1), (u, v, t_2), \dots, (u, v, t_n)\} \subset \mathcal{E}$, where the timestamps are ordered as $0 < t_1 < t_2 < \dots < t_n$. This sequence indicates that nodes u and v have interacted at least once at each corresponding timestamp. Two nodes that interact are referred to as neighbors. The symbols and their definitions are summarized in Table 2, and this notation will be used in the subsequent sections.

3.2. Temporal link prediction

We aim to learn a model that given a pair of nodes with a specific timestamp t , we aim to predict whether the two nodes are connected at t based on all the available historical data. Note that we are not only concerned with the prediction of links between nodes seen during training. We also expect the model to predict links between nodes that are never seen for inductive evaluation. For a more reliable comparison, we use the three strategies in [35] (i.e., random, historical, and inductive negative sampling strategies) to comprehensively evaluate the performance of the model on the temporal link prediction task.

3.3. Transformer

A Transformer block relies heavily on a multi-head attention mechanism to learn the context-aware representation for sequences, which can be defined as:

$$O = \text{MultiHead}(Q, K, V) = W_O \text{Concat}(O_1, O_2, \dots, O_h), \quad (1)$$

where W_O is a trainable parameter. h is the number of heads. O_i is computed as:

$$O_i = \text{Attention}(Q_i, K_i, V_i) = \text{SoftMax}\left(\frac{Q_i K_i^T}{\sqrt{d}}\right) V_i, \quad (2)$$

where $\text{Attention}(\cdot, \cdot, \cdot)$ is the scaled dot-product attention. As input to the Transformer, an element in a sequence is represented by an embedding vector. The multi-head attention mechanism works by injecting the Positional Embedding into the element embeddings to be aware of element orders in a sequence.

4. Proposed method

The framework of our TGFormer is shown in Fig. 1, which employs a Series Transformer as the backbone. Given an interaction (u, v, t) , we first extract historical interactions of source node u and destination node v before timestamp t and obtain two interaction sequences S_u^t and S_v^t . Next, computing the encodings of neighbors, links, time intervals, and interaction frequency for each sequence. Then, we concat each encoding sequence feed into the Series Transformer for capturing long-term temporal dependencies and periodic dependencies. Finally, the outputs of the Transformer adopt an adaptive readout function to derive time-aware representations of u and v at timestamp t (i.e., h_u^t and h_v^t), which can be applied in various downstream tasks like temporal link prediction.

4.1. Extract layer

In our initial endeavor of predicting the connection between the nodal elements u and v at a specific timestamp t , we first affix each node with a self-loop at the specific time t . This maneuver aims to enhance the correlation of the node to its inherent characteristics. Subsequently, we extract L interactions for both nodes, ordered based on temporal proximity, which could interact with the same node at different timestamps. If a node has fewer than L historical neighbors, zero-padding is employed to reconcile the deficit. This process culminates in the successful transmutation of the intricate problem of temporal graph learning into a more comprehensible paradigm - a time series analysis problem. Mathematically, given the predictive interaction (u, v, t) , for the source node u and the target node v , we exact the series of interactions encompassing nodes u and v , expressed as $S_u^t = \{(u, u', t') | t' < t\} \cup \{(u, u, t)\}$ and $S_v^t = \{(v, v', t') | t' < t\} \cup \{(v, v, t)\}$, respectively. Note, $(u, u', t') \in \mathcal{E}, (v, v', t') \in \mathcal{E}$, and $|S_u^t| = |S_v^t| = L$.

4.2. Encoder layer

In this section, we exposition details about how to convert the extracted interactions from temporal graph learning to time series analysis. These interactions, construed as temporally continuous events, embody four encodings: node, link, time, and frequency. Together, these constitute the comprehensive representation of X_*^t , where $*$ refers to either of the nodes u or v .

4.2.1. Node/edge encoding

In the realm of temporal graphs, both vertices (nodes) and interactions (edges or links) often harbor concomitant features. To delineate the embeddings allied with interactions, it is imperative to harvest the inherent traits of proximate nodes and edges considering the sequence, designated as S_*^t . Aligning our methodology with established approaches in the literature [11], we adopt a schema to encode nodes and links correspondingly as $X_{*,N}^t \in \mathbb{R}^{L \times d_N}$ and $X_{*,E}^t \in \mathbb{R}^{L \times d_E}$, wherein d_N and d_E typify the dimensions associated with the specific embeddings of nodes and edges, respectively. The specific schema is defined as:

$$X_{*,N}^t = \text{MLP}(x_{*,N}^t), X_{*,E}^t = \text{MLP}(x_{*,E}^t), \quad (3)$$

where $x_{*,N}^t, x_{*,E}^t$ represents the node $*$ interact nodes' features and the interact link features of S_*^t . If the original features do not exist, they are all set to zero.

4.2.2. Time encoding

Our schema for time-encoding deploys a cosine function, denoted as $\cos(\Delta t \omega)$, where $\omega = \{\alpha^{-(i-1)/\beta}\}_{i=1}^{d_T}$. This function is tasked with transliterating static timestamps into vectors represented as $X_{*,T}^t \in \mathbb{R}^{L \times d_T}$. Here, the symbol d_T signifies the dimensionality of time embeddings, and α and β function as adjustable parameters. Notably, these parameters ensure that the value of $t_{\max} \times \alpha^{-(i-1)/\beta}$ converges to zero when i approaches d_T . So the time encoding can be defined as:

$$X_{*,T}^t = [\cos(\Delta t_1 \omega), \cos(\Delta t_2 \omega), \dots, \cos(\Delta t_L \omega)]^T. \quad (4)$$

In Eq. (4), we adopt relative timestamps for encoding, in preference to absolute ones. In other words, if an interaction occurs at timestamp t_0 , and we are to predict an interaction at a specific timestamp t , the $\Delta t = t - t_0$ is computed. Importantly, ω is held constant throughout the training phase, thereby accelerating the process of model optimization. Furthermore, the adoption of relative time encoding is instrumental in discerning repetitive temporal patterns. This function inspects the temporal gaps between interactions, enabling the construction of a similarity index for comparable timestamps, contributing to the precise temporal differentiation.

4.2.3. Frequency encoding

Most existing methods often undervalue the potential for re-interactions with historical nodes and overlook inherent node correlations. We address this gap via a paradigm shift: the introduction of a node interaction frequency encoding technique. This novel approach posits that the frequency of a node's interaction within a historical sequence signifies its relevance. Our method goes beyond analyzing merely historical interaction sequences and incorporates the frequency of both interaction node occurrence and node pair interaction. This judicious approach effectively captures correlations between two nodes' common interaction nodes within their respective historical interaction sequences.

As illustrated in Fig. 1, we calculate the frequency of each interaction in both S_u^t and S_v^t , represented as $F_u^t \in \mathbb{R}^{L \times 2}$ and $F_v^t \in \mathbb{R}^{L \times 2}$ respectively. This is computed as follows, with the first column denoting the number of times the node appears in S_u^t and the second column denoting the number of times the node appears in S_v^t . Specifically, as shown in Fig. 1, for the interaction nodes $\{u, b, b\}$ and $\{v, d, b\}$ appearing

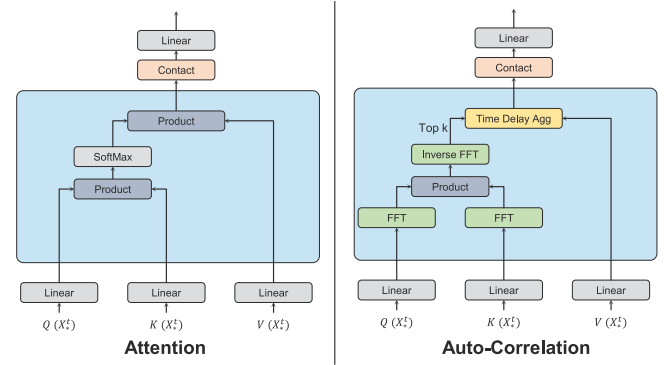


Fig. 2. Attention (left) and ACoM (right). We utilize the Fast Fourier Transform (FFT) to calculate the ACoM, which reflects the time-delay similarities.

in S_u^t and S_v^t , respectively, then $F_u^t = [[1, 2, 2], [0, 1, 1]]$, and $F_v^t = [[0, 0, 2], [1, 1, 1]]$. We then code the frequency of the node pair interaction, deriving node interaction frequency features for u and v , denoted by $X_{*,F}^t \in \mathbb{R}^{L \times d_F}$, wherein d_F means the dimensions of frequency embedding, respectively. This encoding manifests mathematically as follows:

$$X_{*,F}^t = f(F_u^t[:, 0]) + f(F_v^t[:, 1]). \quad (5)$$

We implement a two-layer MLP function with ReLU activation, designated as $f(\cdot)$, to encode the nodes' frequency interaction features.

The frequency Encoding fortifies our hypothesis that nodes with a high frequency of shared historical interaction nodes are likelier to interact in the future. Concurrently, they accommodate the variability of recurrence patterns apparent across different network domains and structures.

4.2.4. Transition to time series analysis

Transitioning next to the realm of time series analysis, we employ the encodings formulated earlier. All these encodings are amalgamated and then projected into a trainable weight space $W_\diamond \in \mathbb{R}^{d_\diamond \times d}$ along with $b_\diamond \in \mathbb{R}^d$ to yield embeddings $X_{u,\diamond}^t \in \mathbb{R}^{L \times d}$ and $X_{v,\diamond}^t \in \mathbb{R}^{L \times d}$. It is imperative to note that the symbol $\diamond$ can be a representative of N, E, T or F . These alignments can be mathematically explained as:

$$\begin{aligned} X_{u,\diamond}^t &= X_{u,\diamond}^t W_\diamond + b_\diamond \in \mathbb{R}^{L \times d}, \\ X_{v,\diamond}^t &= X_{v,\diamond}^t W_\diamond + b_\diamond \in \mathbb{R}^{L \times d}. \end{aligned} \quad (6)$$

We then focus on the concatenation of these encoded nodes which can be calculated as follows:

$$\begin{aligned} X_u^t &= X_{u,N}^t \| X_{u,E}^t \| X_{u,T}^t \| X_{u,F}^t \in \mathbb{R}^{L \times 4d}, \\ X_v^t &= X_{v,N}^t \| X_{v,E}^t \| X_{v,T}^t \| X_{v,F}^t \in \mathbb{R}^{L \times 4d}. \end{aligned} \quad (7)$$

Following this, we treat X_u^t and X_v^t as a time series of length L , thus positioning us for a comprehensive analysis of the time series.

4.3. Series transformer layer

In this section, we will describe how to use the Series Transformer layer, including auto-correlated mechanism and Feed-forward networks.

4.3.1. Auto-correlation mechanism

As depicted in Fig. 2, we propose an ACoM that utilizes node interaction sequences with series-wise connections to optimize information usage. The ACoM uncovers period-based dependencies by calculating the auto-correlation of series and amalgamating similar sub-sequences through time delay aggregation. We briefly describe the procedure of ACoM in Algorithm 1.

Algorithm 1 Auto-Correlation Mechanism

Input: Queries Q , Keys K , Values $V \in \mathbb{R}^{L \times 4d}$; Number of heads h ; Hyper-parameter c .

- 1: $Q, K, V \leftarrow \text{Reshape}(Q), \text{Reshape}(K), \text{Reshape}(V)$
- 2: $Q = \text{FFT}(Q, \text{dim}=0), K = \text{FFT}(K, \text{dim}=0)$
- 3: $\text{Corr} = \text{IFFT}(Q \times \text{Conj}(K), \text{dim}=0)$
- 4: $W_{\text{topk}}, I_{\text{topk}} = \text{Topk}(\text{Corr}, \lfloor c \times \log L \rfloor, \text{dim}=0)$
- 5: $W_{\text{topk}} = \text{Softmax}(W_{\text{topk}}, \text{dim}=0)$
- 6: $\text{Index} = \text{Repeat}(\text{arange}(L))$
- 7: $V = \text{Repeat}(V)$
- 8: $R = [W_{\text{topk}_{i,:}} \times \text{gather}(V, (I_{\text{topk}_{i,:}} + \text{Index})) \text{ for } i \text{ in range}(\lfloor c \times \log L \rfloor)]$
- 9: $R = \text{Sum}(\text{Stack}(R, \text{dim}=0), \text{dim}=0)$
- 10: **Return** R

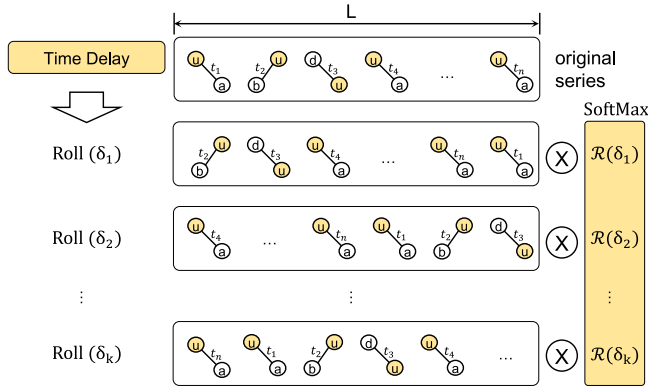


Fig. 3. Time-delay aggregation block. $R(\delta)$ reflects the time-delay similarities. Then the similar sub-processes are rolled to the same index based on selected delay δ and aggregated by $R(\delta)$.

Period-based Dependencies. Inspired by the theory of stochastic processes, we observe that identical phase positions across periods naturally dictate similar sub-processes. Thus, we treat each interaction sequence as a discrete-time process $\{\mathcal{X}_i^t\}_{i=1}^L$, where $\mathcal{X}_i^t$ is the i th row of X^t , i.e., the feature of the i th sampled interaction. Then its auto-correlation, $\mathcal{R}_{\mathcal{X}, \mathcal{X}}(\delta)$, can be computed as follows:

$$\mathcal{R}_{\mathcal{X}, \mathcal{X}}(\delta) = \lim_{L \rightarrow \infty} \frac{1}{L} \sum_{i=1}^L \mathcal{X}_i^t \mathcal{X}_{i-\delta}^t. \quad (8)$$

In this equation, $\mathcal{R}_{\mathcal{X}, \mathcal{X}}(\delta)$ mirrors the time-delay similarity between $\mathcal{X}_i^t$ and the lag series, $\mathcal{X}_{i-\delta}^t$. As shown in Fig. 3, we employ the auto-correlation $R(\delta)$ as the unnormalized confidence of the estimated period length δ . We then select the most probable k period lengths, $\delta_1, \dots, \delta_k$. The resulting period-based dependencies, drawn from the aforementioned estimated periods, can be weighted by the corresponding auto-correlation.

Time Delay Aggregation. Building on these period-based dependencies, it follows that these dependencies interconnect the subsequences across projected periods. In response, we introduce the time delay aggregation block (illustrated in Fig. 3) that allows for the series to roll based on the selected time delay, $\delta_1, \dots, \delta_k$. This operation aligns similar sub-series onto the same phase position of estimated periods, a technique that contrasts with the point-wise dot-product aggregation method used by the attention family. Lastly, we complete the process by incorporating the softmax-normalized confidences to aggregate the sub-sequences.

We initiate our discourse with a single-head scenario involving a time series $\mathcal{X}_i^t$ of length- L . To analogize attention, we use symbols $Q = K = V = X^t$, which are derived from the encoder layer. Thus, it can seamlessly interchange the attention mechanism.

The subsequent step involves introducing the ACoM. This mechanism incorporates several components, with the initial process given by:

$$\delta_1, \dots, \delta_k = \arg \text{Topk}(\mathcal{R}_{Q, K}(\delta)), \quad \delta \in \{1, \dots, L\} \quad (9)$$

where $\arg \text{Topk}(\cdot)$ retrieves the arguments of Topk auto-correlations, and $k = \lfloor c \times \log L \rfloor$. Here, c represents a hyper-parameter. We then normalize these auto-correlations using a SoftMax function, as shown in the equation:

$$\hat{\mathcal{R}}_{Q, K}(\delta_1), \dots, \hat{\mathcal{R}}_{Q, K}(\delta_k) = \text{SoftMax}(\mathcal{R}_{Q, K}(\delta_1), \dots, \mathcal{R}_{Q, K}(\delta_k)). \quad (10)$$

Finally, the ACoM culminates with the equation:

$$\text{ACoM}(Q, K, V) = \sum_{i=1}^k \text{Roll}(V, \delta_i) \hat{\mathcal{R}}_{Q, K}(\delta_i), \quad (11)$$

where $\text{Roll}(\mathcal{X}^t, \delta)$ signifies an operation on $\mathcal{X}^t$ with time delay δ . This operation reinstates any elements shifted beyond the initial position at the end of the sequence.

Advancing to the multi-head version employed in auto-correlation, we handle hidden variables with $4d$ channels and h heads. For each i th head, the query, key, and value are designated as $Q_i, K_i, V_i \in \mathbb{R}^{L \times \frac{4d}{h}}$, $i \in \{1, \dots, h\}$. The operation progression results into:

$$O = \text{MultiHead}(Q, K, V) = W_O \text{Concat}(O_1, O_2, \dots, O_h), \quad (12)$$

where $O_i = \text{ACoM}(Q_i, K_i, V_i)$ from Eq. (11), $W_O \in \mathbb{R}^d$ is a trainable parameter.

Efficient Computation. For period-based dependencies are inherently sparse, linking to sub-processes at identical phase positions within specific periods, prioritizing most probable delays and hence preventing the selection of converse phases. This mechanism, exhibiting computational aptness, aggregates series equivalent to $O(\log L)$, each measuring L in length. The complexity of Eqs. (11) and (12) is $O(L \log L)$, illustrating computational efficiency.

Auto-correlation calculation (Eq. (8)) hinges on Fast Fourier Transforms (FFT), according to the Wiener-Khinchin theorem [21], for a time series $\{\mathcal{X}_i^t\}_{i=1}^L$. The resultant $\mathcal{R}_{\mathcal{X}, \mathcal{X}}(\delta)$ can be deciphered via the following integral equations:

$$S_{\mathcal{X}, \mathcal{X}}(f) = \mathcal{F}(\mathcal{X}_i^t) \mathcal{F}^*(\mathcal{X}_i^t) = \int_{-\infty}^{\infty} \mathcal{X}_i^t e^{-2\pi i f} di \int_{-\infty}^{\infty} \mathcal{X}_i^t e^{-2\pi i f} di, \quad (13)$$

$$\mathcal{R}_{\mathcal{X}, \mathcal{X}}(\delta) = \mathcal{F}^{-1}(S_{\mathcal{X}, \mathcal{X}}(f)) = \int_{-\infty}^{\infty} S_{\mathcal{X}, \mathcal{X}}(f) e^{2\pi i f \delta} df, \quad (14)$$

where δ belongs to the set $\{1, \dots, L\}$, $\mathcal{F}$ signifies FFT, its inverse is $\mathcal{F}^{-1}$, and $*$ denotes the conjugate operation. By using FFT, the auto-correlation of all lags in $\{1, \dots, L\}$ can be computed concurrently, enhancing computational efficiency with complexity at $O(L \log L)$.

4.3.2. Feed-forward network

Our Series Transformer layer emanates from the paradigm of the conventional Transformer encoder, as illustrated in [31]. A unique adaptation within our implementation involves the placement of the layer normalization (LN) before, rather than after [36], the multi-head ACoM, and the feed-forward blocks (FFN). This strategic alteration, now prevalent amongst contemporary Transformer implementations, ensures more effective optimization, as advocated by [37].

Particularly in the context of the FFN sub-layer, we maintain a uniform dimensionality of the input, output, and inner layers, matching the dimensional structure with $4d$. The operational functionality of the Series Transformer layer can thus be formally expounded as follows:

$$Z_*^{(j)} = \text{MultiHead}(Q, K, V) + Z_*^{(j-1)}, \quad (15)$$

$$Z_*^{(j)} = \text{FFN}(\text{LN}(Z_*^{(j)})) + Z_*^{(j)}. \quad (16)$$

where Q, K, V all equal $Z_*^{(j-1)}$. The input of the first layer is $Z_*^0 = X_*^t \in \mathbb{R}^{L \times 4d}$, and the output of the J th layer is denoted by $H_*^t = Z_*^{(J)} \in \mathbb{R}^{L \times 4d}$. $1 \leq j \leq J$ denotes the transformer layer number.

4.4. Adaptive readout

For the output matrix $H_*^t \in \mathbb{R}^{L \times 4d}$ of a node, $H_{*,1}^t \in \mathbb{R}^{1 \times 4d}$ is the token representation of the node interacting with itself and $H_{*,l}^t \in \mathbb{R}^{1 \times 4d}$ is its l th event representation. We calculate the normalized attention coefficients for its l th event:

$$\lambda_l = \frac{\exp((H_{*,1}^t \| H_{*,l}^t) W_a^T)}{\sum_{i=2}^L \exp((H_{*,1}^t \| H_{*,i}^t) W_a^T)}, \quad (17)$$

where $W_a \in \mathbb{R}^{1 \times 8d}$ denotes the learnable projection and $l = 2, \dots, L$. Therefore, the readout function takes the correlation between each event and the node representation into account. The node representation is finally aggregated as follows:

$$h_*^t = H_{*,1}^t + \sum_{l=2}^L \lambda_l H_{*,l}^t. \quad (18)$$

4.5. Loss function

For link prediction loss, we adopt binary cross-entropy loss function, which is defined as:

$$\mathcal{L}_p = - \sum_{i=1}^S (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)), \quad (19)$$

$$\hat{y} = \text{Softmax}(\text{MLP}(\text{RELU}(\text{MLP}(h_u^t \| h_v^t))), \quad (20)$$

where y_i represents the ground-truth label of i th sample and the $\hat{y}_i$ uses two nodes' representation represents the prediction value.

4.6. Complexity analysis

To analyze the time complexity of TGFormer, we first present its pseudo-code as shown in Algorithm 2. For simplicity, the representation dimensions for both input and hidden features are denoted by d , and M signifies the size of temporal edges $|\mathcal{E}|$. The sampling process responsible for acquiring the L most recent neighbors achieves a time complexity of $O(1)$. Modules can be executed in parallel within the encoder layer, where the encoder complexity reaches a maximum of $O(dL)$. Furthermore, the series transformer layer demands significant computational resources; however, we mitigate the time complexity through the use of subsequences (as mentioned in Section 4.3.1), reducing it to $O(dL \log L)$. Consequently, the overall time complexity of TGFormer is at most $O(MdL \log L)$.

Algorithm 2 Training pipeline for TGFormer

Input: A temporal graph $\mathcal{G}$, a node pair (u, v) with a specific timestamp (t) , the neighbor sample number L , maximum training epoch of 200, early stopping strategy with patience = 20.

Output: The probability of the node pair interacting at timestamp t .

```

1: initial patience = 0;
2: for training epoch = 1, 2, 3, ... do
3:   Acquire the  $L$  most recent first-hop interaction neighbors of
     nodes  $u$  and  $v$  from  $\mathcal{G}$  prior to timestamp  $t$  as  $S_u^t$  and  $S_v^t$ ;
4:   for  $S_u^t$  and  $S_v^t$  in parallel do
5:     Obtain node encoding  $X_{*,N}^t$  and edge encoding  $X_{*,E}^t$  from
       Eq. Eq. (3);
6:     Obtain time encoding  $X_{*,T}^t$  from Eq. Eq. (4);
7:     Obtain frequency encoding  $X_{*,F}^t$  from Eq. Eq. (5);
8:      $X_*^t \leftarrow X_{*,N}^t \| X_{*,E}^t \| X_{*,T}^t \| X_{*,F}^t$ ;
9:     Initial  $Z_*^{(0)} \leftarrow X_*^t$ ;
10:    for Series Transformer Layer  $j$  do
11:       $Q, K, V \leftarrow X_*^t$ ;
12:       $Z_*^{(j)} \leftarrow \text{ACoM}(Q, K, V) + Z_*^{(j-1)}$ ;
13:       $Z_*^{(j)} \leftarrow \text{FFN}(\text{LN}(Z_*^{(j)} + Z_*^{(j-1)}))$ ;
14:    end for
15:    Adaptive readout with Eq. Eq. (18);
16:  end for
17:  Conduct link prediction with Eq. Eq. (20);
18:  Compute loss  $\mathcal{L}_p$  with Eq. Eq. (19);
19:  if current epoch's metrics worse than the previous epoch's then
20:    patience = patience + 1
21:  else
22:    Save the model parameters from the current epoch;
23:  end if
24:  if patience = 20 then
25:    Exit training process;
26:  end if
27: end for

```

5. Experiments

5.1. Datasets

We evaluate the performance of TGFormer on temporal link prediction tasks utilizing six publicly available temporal graph datasets: Wikipedia, Reddit, LastFM, Enron, UCI, and CollegeMsg. The statistical characteristics of these datasets are comprehensively detailed in Table 3. Further details of the datasets are shown in Appendix A.

5.2. Baselines

Our model is compared with ten state-of-the-art methods on temporal graphs. They are based on sequential models (JODIE [27], DyRep [38]), memory networks (TGN [11]), random walks (CAWN [12]), attention mechanisms (TGAT [28], TCL [39], DyGFormer [16]), and others (EdgeBank [35], GraphMixer [25], FreeDyG [40]). Further details of baselines are shown in Appendix B.

5.3. Evaluation tasks and metrics

We closely follow [16] by evaluating the model performance for temporal link prediction, which entails forecasting the probability of a link formation between two specified nodes at a given timestamp. This evaluation is conducted under two distinct settings: the transductive setting, which aims to predict future links among nodes observed during the training phase, and the inductive setting, which endeavors to predict future links involving previously unseen nodes. To this

Table 3

Statistics of the datasets.

Datasets	Domains	#Nodes	#Links	#Node & Link features	Bipartite	Time granularity	Duration
Wikipedia	Social	9227	157,474	- & 172	True	Unix timestamp	1 month
Reddit	Social	10,984	672,447	- & 172	True	Unix timestamp	1 month
LastFM	Interaction	1980	1,293,103	- & -	True	Unix timestamp	1 month
Enron	Social	184	125,235	- & -	False	Unix timestamp	3 years
UCI	Social	1899	59,835	- & -	False	Unix timestamp	196 days
CollegeMsg	Social	1899	59,835	- & -	False	Unix timestamp	193 days

Table 4AP (%) for *transductive* temporal link prediction with random, historical, and inductive negative sampling strategies. NSS is the abbreviation of Negative Sampling Strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	EdgeBank	TCL	GraphMixer	DyGFormer	FreeDyG	TGFormer
Random	Wikipedia	96.50 ± 0.14	94.86 ± 0.06	96.94 ± 0.06	98.45 ± 0.06	98.76 ± 0.03	90.37 ± 0.00	96.47 ± 0.16	97.25 ± 0.03	99.03 ± 0.02	99.26 ± 0.01	99.79 ± 0.10
	Reddit	98.31 ± 0.14	98.22 ± 0.04	98.52 ± 0.02	98.63 ± 0.06	99.11 ± 0.01	94.86 ± 0.00	97.53 ± 0.02	97.31 ± 0.01	99.22 ± 0.01	99.48 ± 0.01	99.86 ± 0.01
	LastFM	70.85 ± 2.13	71.92 ± 2.21	73.42 ± 0.21	77.07 ± 3.97	86.99 ± 0.06	79.29 ± 0.00	67.27 ± 2.16	75.61 ± 0.24	93.00 ± 0.12	92.15 ± 0.16	96.87 ± 0.34
	Enron	84.77 ± 0.30	82.38 ± 3.36	71.12 ± 0.97	86.53 ± 1.11	89.56 ± 0.09	83.53 ± 0.00	79.70 ± 0.71	82.25 ± 0.16	92.47 ± 0.12	92.51 ± 0.05	97.14 ± 0.72
	UCI	89.43 ± 1.09	65.14 ± 2.30	79.63 ± 0.70	92.34 ± 1.04	95.18 ± 0.06	76.20 ± 0.00	89.57 ± 1.63	93.25 ± 0.57	95.79 ± 0.17	96.28 ± 0.11	99.09 ± 0.17
	CollegeMsg	75.41 ± 1.82	56.92 ± 6.03	80.27 ± 0.29	92.62 ± 0.99	95.86 ± 0.06	76.42 ± 0.00	83.64 ± 0.10	93.04 ± 0.29	95.79 ± 0.02	95.81 ± 0.22	99.17 ± 0.22
	Avg. rank	8.17	9.5	8.17	5.5	3.67	8.83	8.83	7	3	2.33	1
Historical	Wikipedia	83.01 ± 0.66	79.93 ± 0.56	87.38 ± 0.22	86.86 ± 0.33	71.21 ± 1.67	73.35 ± 0.00	89.05 ± 0.39	90.90 ± 0.10	82.23 ± 2.54	91.59 ± 0.57	92.47 ± 0.19
	Reddit	80.03 ± 0.36	79.83 ± 0.31	79.55 ± 0.20	81.22 ± 0.61	80.82 ± 0.45	73.59 ± 0.00	77.14 ± 0.16	78.44 ± 0.18	81.57 ± 0.67	85.67 ± 1.01	84.44 ± 1.49
	LastFM	74.35 ± 3.81	74.92 ± 2.46	71.59 ± 0.24	76.87 ± 4.64	69.86 ± 0.43	73.03 ± 0.00	59.30 ± 2.31	72.47 ± 0.49	81.57 ± 0.48	79.71 ± 0.51	84.52 ± 0.67
	Enron	69.85 ± 2.70	71.19 ± 2.76	64.07 ± 1.05	73.91 ± 1.76	64.73 ± 0.36	76.53 ± 0.00	70.66 ± 0.39	77.98 ± 0.92	75.63 ± 0.73	78.87 ± 0.82	82.64 ± 1.39
	UCI	75.24 ± 5.80	55.10 ± 3.14	68.27 ± 1.37	80.43 ± 2.12	65.30 ± 0.43	65.50 ± 0.00	80.25 ± 2.74	84.11 ± 1.35	82.17 ± 0.82	86.10 ± 1.19	86.12 ± 1.50
	CollegeMsg	64.41 ± 6.52	47.73 ± 0.97	68.18 ± 1.04	80.65 ± 1.13	84.54 ± 0.11	44.16 ± 0.00	68.53 ± 0.08	83.93 ± 0.09	80.93 ± 0.37	89.11 ± 0.34	90.02 ± 1.19
	Avg. rank	7.33	8.17	8.17	5.17	8.17	8.67	7.67	5	4.5	2	1.17
Inductive	Wikipedia	75.65 ± 0.79	70.21 ± 1.58	87.00 ± 0.16	85.62 ± 0.44	74.06 ± 2.62	80.63 ± 0.00	86.76 ± 0.72	88.59 ± 0.17	78.29 ± 5.38	90.05 ± 0.79	93.10 ± 0.71
	Reddit	86.98 ± 0.16	86.30 ± 0.26	89.59 ± 0.24	88.10 ± 0.24	91.67 ± 0.24	85.48 ± 0.00	87.45 ± 0.29	85.26 ± 0.11	91.11 ± 0.40	90.74 ± 0.17	92.37 ± 1.64
	LastFM	62.67 ± 4.49	64.41 ± 2.70	71.13 ± 0.17	65.95 ± 5.98	67.48 ± 0.77	75.49 ± 0.00	58.21 ± 0.89	68.12 ± 0.33	73.97 ± 0.50	72.19 ± 0.24	74.01 ± 1.35
	Enron	68.96 ± 0.98	67.79 ± 1.53	63.94 ± 1.36	70.89 ± 2.72	75.15 ± 0.58	73.89 ± 0.00	71.29 ± 0.32	75.01 ± 0.79	77.41 ± 0.89	77.81 ± 0.65	84.34 ± 1.74
	UCI	65.99 ± 1.40	54.79 ± 1.76	68.67 ± 0.84	70.94 ± 0.71	64.61 ± 0.48	57.43 ± 0.00	76.01 ± 1.11	80.10 ± 0.51	72.25 ± 1.71	82.35 ± 0.73	80.70 ± 1.16
	CollegeMsg	53.49 ± 0.53	54.13 ± 1.64	68.54 ± 0.85	71.90 ± 0.06	75.12 ± 0.09	43.49 ± 0.00	68.70 ± 0.02	79.17 ± 0.09	70.44 ± 0.83	81.42 ± 1.15	80.15 ± 1.72
	Avg. rank	9	9.83	6.67	6.5	6	7.5	6.83	5.17	4.67	2.33	1.5

Table 5AUC-ROC (%) for *transductive* temporal link prediction with random, historical, and inductive negative sampling strategies. NSS is the abbreviation of Negative Sampling Strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	EdgeBank	TCL	GraphMixer	DyGFormer	FreeDyG	TGFormer
Random	Wikipedia	96.33 ± 0.07	94.37 ± 0.09	96.67 ± 0.07	98.37 ± 0.07	98.54 ± 0.04	90.78 ± 0.00	95.84 ± 0.18	96.92 ± 0.03	98.91 ± 0.02	99.41 ± 0.01	99.77 ± 0.10
	Reddit	98.31 ± 0.05	98.17 ± 0.05	98.47 ± 0.02	98.60 ± 0.06	99.01 ± 0.01	95.37 ± 0.00	97.42 ± 0.02	97.17 ± 0.02	99.15 ± 0.01	99.50 ± 0.01	99.82 ± 0.01
	LastFM	70.49 ± 1.66	71.16 ± 1.89	71.59 ± 0.18	78.47 ± 2.94	85.92 ± 0.10	83.77 ± 0.00	64.06 ± 1.16	73.53 ± 0.12	93.05 ± 0.10	93.42 ± 0.15	96.85 ± 0.44
	Enron	87.96 ± 0.52	84.89 ± 3.00	68.89 ± 1.10	88.32 ± 0.99	90.45 ± 0.14	87.05 ± 0.00	75.74 ± 0.72	84.38 ± 0.21	93.33 ± 0.13	94.01 ± 0.11	97.13 ± 0.90
	UCI	90.44 ± 0.49	68.77 ± 2.34	78.53 ± 0.74	92.03 ± 1.13	93.87 ± 0.08	77.30 ± 0.00	87.82 ± 1.36	91.81 ± 0.67	94.49 ± 0.26	95.00 ± 0.21	98.68 ± 0.30
	CollegeMsg	78.87 ± 1.71	61.72 ± 6.95	79.25 ± 0.76	92.39 ± 1.00	94.63 ± 0.11	77.32 ± 0.00	83.32 ± 0.08	91.71 ± 0.28	94.52 ± 0.03	95.07 ± 0.20	98.88 ± 0.30
	Avg. rank	7.83	9.5	8.17	5.17	3.83	9	9	7.33	3.17	2	1
Historical	Wikipedia	80.77 ± 0.73	77.74 ± 0.33	82.87 ± 0.22	82.74 ± 0.32	67.84 ± 0.64	77.27 ± 0.00	85.76 ± 0.46	87.68 ± 0.17	78.80 ± 1.95	82.78 ± 0.30	92.07 ± 0.33
	Reddit	80.52 ± 0.32	80.15 ± 0.18	79.33 ± 0.16	81.11 ± 0.19	80.27 ± 0.30	78.58 ± 0.00	76.49 ± 0.16	77.80 ± 0.12	80.54 ± 0.29	85.92 ± 0.10	83.08 ± 2.43
	LastFM	75.22 ± 2.36	74.65 ± 1.98	64.27 ± 0.26	77.97 ± 3.04	67.88 ± 0.24	78.09 ± 0.00	47.24 ± 3.13	64.21 ± 0.73	78.78 ± 0.35	75.74 ± 0.72	82.39 ± 0.67
	Enron	75.39 ± 2.37	74.69 ± 3.55	61.85 ± 1.43	77.09 ± 2.22	65.10 ± 0.34	79.59 ± 0.00	67.95 ± 0.88	75.27 ± 1.14	76.55 ± 0.52	75.74 ± 0.72	80.59 ± 1.47
	UCI	78.64 ± 3.50	57.91 ± 3.12	58.89 ± 1.57	77.25 ± 2.68	57.86 ± 0.15	69.56 ± 0.00	72.25 ± 3.46	77.54 ± 2.02	76.97 ± 0.24	80.38 ± 0.26	83.80 ± 2.67
	CollegeMsg	66.92 ± 5.00	49.37 ± 1.37	58.19 ± 1.11	77.65 ± 1.40	79.45 ± 0.15	34.64 ± 0.00	58.55 ± 0.18	77.50 ± 0.16	76.25 ± 0.12	84.83 ± 0.40	88.58 ± 1.10
	Avg. rank	5.67	8.5	8.33	4.33	8.17	7.33	8.17	6.33	5.17	2.67	1.33
Inductive	Wikipedia	70.96 ± 0.78	67.36 ± 0.96	81.93 ± 0.22	80.97 ± 0.31	70.95 ± 0.95	81.73 ± 0.00	82.19 ± 0.48	84.28 ± 0.30	75.09 ± 3.70	82.74 ± 0.32	91.01 ± 0.76
	Reddit	83.51 ± 0.15	82.90 ± 0.31	87.13 ± 0.20	84.56 ± 0.24	88.04 ± 0.29	85.93 ± 0.00	84.67 ± 0.29	82.21 ± 0.13	86.23 ± 0.51	84.38 ± 0.21	89.42 ± 2.17
	LastFM	61.32 ± 3.49	62.15 ± 2.12	63.99 ± 0.21	65.46 ± 4.27	67.92 ± 0.44	77.37 ± 0.00	46.93 ± 2.59	60.22 ± 0.32	69.25 ± 0.36	72.30 ± 0.59	70.54 ± 0.67
	Enron	70.92 ± 1.05	68.73 ± 1.34	60.45 ± 2.12	71.34 ± 2.46	75.17 ± 0.50	75.00 ± 0.00	67.64 ± 0.86	71.53 ± 0.85	74.07 ± 0.64	77.27 ± 0.61	81.19 ± 1.21
	UCI	64.14 ± 1.26	54.25 ± 2.01	60.80 ± 1.01	64.11 ± 1.04	58.06 ± 0.26	58.03 ± 0.00	70.05 ± 1.86	74.59 ± 0.74	65.96 ± 1.18	75.39 ± 0.57	75.98 ± 2.90
	CollegeMsg	53.34 ± 0.22	53.13 ± 2.16	59.84 ± 1.16	65.58 ± 0.42	69.28 ± 0.24	30.53 ± 0.00	59.67 ± 0.16	74.06 ± 0.25	64.77 ± 0.13	76.32 ± 0.67	76.92 ± 1.31
	Avg. rank	8.33	9.83	6.83	6.5	5.5	6.17	7.17	5.83	5.33	3.17	1.33

end, a multi-layer perceptron is utilized, which takes the concatenated representations of the two nodes as input and outputs the likelihood of a link. Evaluation metrics employed include Average Precision (AP) and Area Under the Receiver Operating Characteristic Curve (AUC-ROC). For the node classification task, AUC-ROC is adopted as the performance metric. All results presented are the aggregate of 10 independent experimental runs.

To provide a more comprehensive comparison, we employ three negative sampling strategies for evaluating temporal link prediction, namely, random, historical, and inductive negative sampling, as proposed in [35]. These strategies differ in the manner in which negative edges are selected, with the latter two imposing more stringent constraints. Specifically, random negative sampling selects negative edges by randomly pairing nodes from the entire graph, ensuring broad coverage of possible negative interactions. In contrast, historical negative sampling restricts negative edges to those that have previously

appeared in past timestamps but are absent in the current step, thereby assessing a model's ability to predict edge reoccurrence patterns. Lastly, inductive negative sampling samples negative edges exclusively from previously unseen edges during training, focusing on the model's capacity to generalize to novel structures. For all tasks, datasets are chronologically split into 70%, 15%, and 15% for training, validation, and testing, respectively. Hyperparameter configurations for baseline models strictly follow those outlined in their respective publications, consistent with [16].

5.4. Implementation details

For both tasks, all models undergo a training regimen of up to 100 epochs, incorporating the early stopping strategy with a patience parameter set to 10. The model achieving optimal performance on the validation set is selected for subsequent testing. The Adam optimizer

Table 6AP (%) for *inductive* temporal link prediction with random, historical, and inductive negative sampling strategies. NSS is the abbreviation of Negative Sampling Strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	TCL	GraphMixer	DyGFormer	FreeDyG	TGFormer
Random	Wikipedia	94.82 ± 0.20	92.43 ± 0.37	96.22 ± 0.07	97.83 ± 0.04	98.24 ± 0.03	96.22 ± 0.17	96.65 ± 0.02	98.59 ± 0.03	<u>98.97 ± 0.01</u>	99.27 ± 0.25
	Reddit	96.50 ± 0.13	96.09 ± 0.11	97.09 ± 0.04	97.50 ± 0.07	98.62 ± 0.01	94.09 ± 0.07	95.26 ± 0.02	98.84 ± 0.02	<u>98.91 ± 0.01</u>	99.78 ± 0.04
	LastFM	81.61 ± 3.82	83.02 ± 1.48	78.63 ± 0.31	81.45 ± 4.29	89.42 ± 0.07	73.53 ± 1.66	82.11 ± 0.42	94.23 ± 0.09	<u>94.89 ± 0.01</u>	97.97 ± 0.37
	Enron	80.72 ± 1.39	74.55 ± 3.95	67.05 ± 1.51	77.94 ± 1.02	86.35 ± 0.51	76.14 ± 0.79	75.88 ± 0.48	<u>89.76 ± 0.34</u>	89.69 ± 0.17	91.88 ± 1.26
	UCI	79.86 ± 1.48	57.48 ± 1.87	79.54 ± 0.48	88.12 ± 2.05	92.73 ± 0.06	87.36 ± 2.03	91.19 ± 0.42	94.54 ± 0.12	<u>94.85 ± 0.10</u>	99.16 ± 0.15
	CollegeMsg	63.30 ± 1.26	52.72 ± 2.58	79.58 ± 0.47	88.07 ± 1.44	94.44 ± 0.05	81.18 ± 0.04	90.94 ± 0.22	94.36 ± 0.12	93.99 ± 0.41	97.73 ± 0.22
	Avg. rank	7.5	8.67	8.17	6	3.67	8.17	6.5	2.83	<u>2.5</u>	1
Historical	Wikipedia	68.69 ± 0.39	62.18 ± 1.27	<u>84.17 ± 0.22</u>	81.76 ± 0.32	67.27 ± 1.63	82.20 ± 2.18	87.60 ± 0.30	71.42 ± 4.43	82.78 ± 0.30	78.69 ± 1.25
	Reddit	62.34 ± 0.54	61.60 ± 0.72	63.47 ± 0.36	64.85 ± 0.85	63.67 ± 0.41	60.83 ± 0.25	64.50 ± 0.26	65.37 ± 0.60	<u>66.02 ± 0.41</u>	72.95 ± 1.13
	LastFM	70.39 ± 4.31	71.45 ± 1.76	75.27 ± 0.25	66.65 ± 6.11	71.33 ± 0.47	65.78 ± 0.65	76.42 ± 0.22	76.35 ± 0.52	77.28 ± 0.21	<u>76.81 ± 0.79</u>
	Enron	65.86 ± 3.71	62.08 ± 2.27	61.40 ± 1.31	62.91 ± 1.16	60.70 ± 0.36	67.11 ± 0.62	72.37 ± 1.37	67.07 ± 0.62	<u>73.01 ± 0.88</u>	76.77 ± 1.01
	UCI	63.11 ± 2.27	52.47 ± 2.06	70.52 ± 0.93	70.78 ± 0.78	64.54 ± 0.47	76.71 ± 1.00	<u>81.66 ± 0.49</u>	72.13 ± 1.87	82.35 ± 0.39	77.24 ± 1.59
	CollegeMsg	50.51 ± 0.75	54.43 ± 1.79	70.50 ± 1.18	71.60 ± 0.31	74.14 ± 0.17	69.80 ± 0.23	<u>80.15 ± 0.18</u>	69.59 ± 1.25	83.31 ± 0.78	77.80 ± 1.11
	Avg. rank	8.17	8.67	6	6	7.33	6.5	2.67	5.33	1.67	<u>2.67</u>
Inductive	Wikipedia	68.70 ± 0.39	62.19 ± 1.28	84.17 ± 0.22	81.77 ± 0.32	67.24 ± 1.63	82.20 ± 2.18	87.60 ± 0.29	71.42 ± 4.43	<u>87.54 ± 0.26</u>	78.13 ± 1.56
	Reddit	62.32 ± 0.54	61.58 ± 0.72	63.40 ± 0.36	64.84 ± 0.84	63.65 ± 0.41	60.81 ± 0.26	64.49 ± 0.25	65.35 ± 0.60	64.98 ± 0.20	72.95 ± 1.13
	LastFM	70.39 ± 4.31	71.45 ± 1.75	76.28 ± 0.25	69.46 ± 4.65	71.33 ± 0.47	65.78 ± 0.65	<u>76.42 ± 0.22</u>	76.35 ± 0.52	77.01 ± 0.43	74.21 ± 1.02
	Enron	65.86 ± 3.71	62.08 ± 2.27	61.40 ± 1.30	62.90 ± 1.16	60.72 ± 0.36	67.11 ± 0.62	72.37 ± 1.38	67.07 ± 0.62	72.85 ± 0.81	76.77 ± 1.02
	UCI	63.16 ± 2.27	52.47 ± 2.09	70.49 ± 0.93	70.73 ± 0.79	64.54 ± 0.47	76.65 ± 0.99	<u>81.64 ± 0.49</u>	72.13 ± 1.86	82.06 ± 0.58	77.19 ± 1.09
	CollegeMsg	50.57 ± 0.76	54.47 ± 1.81	70.50 ± 1.19	71.63 ± 0.31	74.11 ± 0.17	69.80 ± 0.24	<u>80.13 ± 0.18</u>	69.55 ± 1.27	83.33 ± 0.79	77.72 ± 1.11
	Avg. rank	8.17	8.67	6	6	7.33	6.5	<u>2.5</u>	5	1.67	3.17

Table 7AUC-ROC (%) for *inductive* temporal link prediction with random, historical, and inductive negative sampling strategies. NSS is the abbreviation of Negative Sampling Strategies.

NSS	Datasets	JODIE	DyRep	TGAT	TGN	CAWN	TCL	GraphMixer	DyGFormer	FreeDyG	TGFormer
Random	Wikipedia	94.33 ± 0.27	91.49 ± 0.45	95.90 ± 0.09	97.72 ± 0.03	98.03 ± 0.04	95.57 ± 0.20	96.30 ± 0.04	98.48 ± 0.03	<u>99.01 ± 0.02</u>	99.15 ± 0.28
	Reddit	96.52 ± 0.13	96.05 ± 0.12	96.98 ± 0.04	97.39 ± 0.07	98.42 ± 0.02	93.80 ± 0.07	94.97 ± 0.05	98.71 ± 0.01	<u>98.84 ± 0.01</u>	99.69 ± 0.06
	LastFM	81.13 ± 3.39	82.24 ± 1.51	76.99 ± 0.29	82.61 ± 3.15	87.82 ± 0.12	70.84 ± 0.85	80.37 ± 0.18	94.08 ± 0.08	<u>94.32 ± 0.03</u>	97.93 ± 0.37
	Enron	81.96 ± 1.34	76.34 ± 4.20	64.63 ± 1.74	78.83 ± 1.11	87.02 ± 0.50	72.33 ± 0.99	76.51 ± 0.71	90.69 ± 0.26	89.51 ± 0.20	89.92 ± 1.45
	UCI	78.80 ± 0.94	58.08 ± 1.81	77.64 ± 0.38	86.68 ± 2.29	90.40 ± 0.11	84.49 ± 1.82	89.30 ± 0.57	92.63 ± 0.13	<u>93.01 ± 0.08</u>	98.84 ± 0.22
	CollegeMsg	66.00 ± 2.05	54.09 ± 3.53	77.00 ± 0.17	86.73 ± 1.82	92.29 ± 0.06	78.76 ± 0.05	88.40 ± 0.18	92.21 ± 0.13	<u>92.74 ± 0.34</u>	96.49 ± 0.31
	Avg. rank	7.5	8.67	8.17	5.5	3.83	8.5	6.67	2.83	<u>2.17</u>	1.17
Historical	Wikipedia	61.86 ± 0.53	57.54 ± 1.09	78.38 ± 0.20	75.75 ± 0.29	62.04 ± 0.65	79.79 ± 0.96	82.87 ± 0.21	68.33 ± 2.82	<u>82.08 ± 0.32</u>	76.75 ± 1.67
	Reddit	61.69 ± 0.39	60.45 ± 0.37	64.43 ± 0.27	64.55 ± 0.50	64.94 ± 0.21	61.43 ± 0.26	64.27 ± 0.13	64.81 ± 0.25	<u>66.79 ± 0.31</u>	69.26 ± 2.38
	LastFM	68.44 ± 3.26	68.79 ± 1.08	69.89 ± 0.28	66.99 ± 5.62	67.69 ± 0.24	55.88 ± 1.85	70.07 ± 0.20	<u>70.73 ± 0.37</u>	72.63 ± 0.16	70.33 ± 1.02
	Enron	65.32 ± 3.57	61.50 ± 2.50	57.84 ± 2.18	62.68 ± 1.09	62.25 ± 0.40	64.06 ± 1.02	68.20 ± 1.62	65.78 ± 0.42	70.09 ± 0.65	68.79 ± 1.58
	UCI	60.24 ± 1.94	51.25 ± 2.37	62.32 ± 1.18	62.69 ± 0.90	56.39 ± 0.10	70.46 ± 1.94	<u>75.98 ± 0.84</u>	65.55 ± 1.01	<u>76.01 ± 0.75</u>	67.72 ± 1.97
	CollegeMsg	48.57 ± 1.27	52.31 ± 1.53	61.53 ± 1.45	63.89 ± 0.71	67.77 ± 0.09	60.05 ± 0.08	<u>74.54 ± 0.16</u>	63.15 ± 0.44	<u>77.55 ± 0.52</u>	74.18 ± 0.49
	Avg. rank	7.83	9	6.5	6.33	6.67	6.5	3.17	4.67	1.33	<u>3</u>
Inductive	Wikipedia	61.87 ± 0.53	57.54 ± 1.09	78.38 ± 0.20	75.76 ± 0.29	62.02 ± 0.65	79.79 ± 0.96	<u>82.88 ± 0.21</u>	68.33 ± 2.82	83.17 ± 0.31	76.04 ± 2.08
	Reddit	61.69 ± 0.39	60.44 ± 0.37	64.39 ± 0.27	64.55 ± 0.50	<u>64.91 ± 0.21</u>	61.36 ± 0.26	64.27 ± 0.13	64.80 ± 0.25	64.51 ± 0.19	69.25 ± 2.38
	LastFM	68.44 ± 3.26	68.79 ± 1.08	69.89 ± 0.28	66.99 ± 5.61	67.68 ± 0.24	55.88 ± 1.85	70.07 ± 0.20	70.73 ± 0.37	<u>71.42 ± 0.33</u>	73.21 ± 1.42
	Enron	65.32 ± 3.57	61.50 ± 2.50	57.83 ± 2.18	62.68 ± 1.09	62.27 ± 0.40	64.05 ± 1.02	68.19 ± 1.63	65.79 ± 0.42	<u>68.79 ± 0.91</u>	72.79 ± 1.59
	UCI	60.27 ± 1.94	51.26 ± 2.40	62.29 ± 1.17	62.66 ± 0.91	56.39 ± 0.11	70.42 ± 1.93	75.97 ± 0.85	65.58 ± 1.00	<u>73.41 ± 0.88</u>	72.78 ± 1.34
	CollegeMsg	48.64 ± 1.26	52.36 ± 1.53	61.51 ± 1.47	63.93 ± 0.70	67.72 ± 0.09	60.05 ± 0.10	74.53 ± 0.16	63.14 ± 0.44	<u>77.58 ± 0.53</u>	73.45 ± 0.51
	Avg. rank	7.83	9	6.5	6.17	6.5	6.67	3.17	4.67	2.17	<u>2.33</u>

is uniformly employed across all models, and uses supervised binary cross-entropy loss as the objective function, maintaining a consistent learning rate of 0.00001 and a batch size of 200. We conduct our experiments on a machine with the Intel(R) Xeon(R) Gold 6330 CPU @ 2.00 GHz with 256 GiB RAM, and four NVIDIA 3090 GPU cards, which is implemented in Python 3.9 with Pytorch. See our code for more details are in <https://github.com/hjchen-hdu/TGFormer>.

5.5. Link prediction

We compare our TGFormer with the previous state-of-the-art in both transductive and inductive link prediction using the AP and AUC-ROC metrics. To provide a more comprehensive study of our TGFormer, we present results for all three negative sampling strategies. We put the *transductive* results in Table 4 and Table 5 and *inductive* results in Tables 6 and 7, respectively. The best and second-best results are marked in bold and underlined fonts. Note that EdgeBank [35] can only evaluate transductive temporal link prediction and therefore does not give its results in the inductive setting. From the above table, we can see that TGFormer outperforms the existing methods in most cases, which is far superior to the second one.

We summarize the superiority of TGFormer in two aspects. First, the transformer architecture allows TGFormer to process longer histories and capture long-term temporal dependencies effectively. As illustrated in Fig. 6, the input sequence lengths handled by TGFormer are significantly longer than those managed by baseline across most datasets,

Table 8

AUC-ROC (%) for the transductive node classification on Wikipedia and Reddit.

Methods	Wikipedia	Reddit	Avg. rank
JODIE	88.99 ± 1.05	60.37 ± 2.58	5
DyRep	86.39 ± 0.98	63.72 ± 1.32	5.5
TGAT	84.09 ± 1.27	70.04 ± 1.09	4.5
TGN	86.38 ± 2.34	63.27 ± 0.90	6.5
CAWN	84.88 ± 1.33	66.34 ± 1.78	6
TCL	77.83 ± 2.13	68.87 ± 2.15	5.5
GraphMixer	86.80 ± 0.79	64.22 ± 3.32	5.5
DyGFormer	<u>87.44 ± 1.08</u>	68.00 ± 1.74	2.5
TGFormer	<u>86.40 ± 2.24</u>	66.41 ± 2.16	<u>4</u>

underscoring TGFormer’s superior aptitude in leveraging longer sequences. Second, benefiting from capturing periodic temporal patterns, especially for Wikipedia and Reddit in the case of harder negative sampling, TGFormer also has a small performance degradation compared to the other approaches.

5.6. Node classification

For the node classification task, we adhere to the evaluation protocols established by DyGFormer [16]. The objective of this task is to predict the state label of the source node, given the node and future timestamps. Specifically, we employ the model obtained from

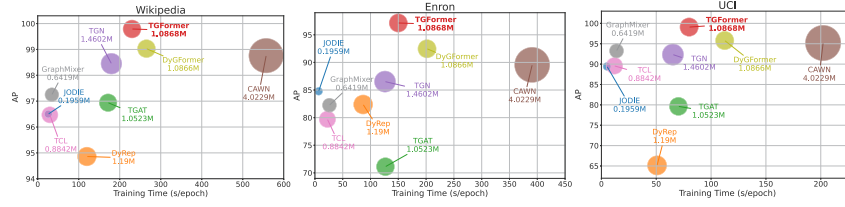


Fig. 4. Comparison of model performance, parameter size and training time per epoch on Wikipedia, Reddit, and UCI.

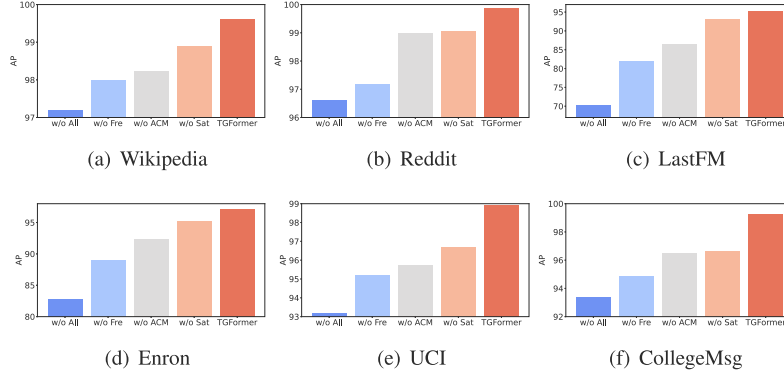


Fig. 5. Ablation study in the transductive setting with the random negative sampling strategy. AP (%) values are reported.

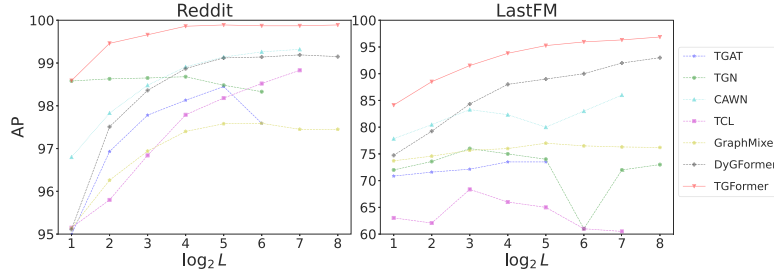


Fig. 6. The performance of various methods on Reddit and LastFM datasets across different historical lengths L .

the preceding transductive link prediction as the pre-trained model for node classification. A classifier decoder, such as a three-layer MLP, is then trained separately for the node classification task. We evaluate this task on two datasets with dynamic node labels, namely Wikipedia and Reddit, while excluding other datasets due to the absence of node labels.

Table 8 presents the results of our method compared to the baseline for the node classification task. Although our method does not achieve optimal performance, it remains competitive. The performance limitations can be attributed to the model's emphasis on node interactions, such as periodic temporal pattern dependencies while placing less focus on individual node states, which adversely impacts its effectiveness in node classification.

5.7. Efficiency analysis

To compare the efficiency of the evaluated models, we conducted a comprehensive analysis encompassing performance in transductive link prediction using AP metrics, training time per epoch, and the number of trainable parameters between TGFormer and baseline methods across the Wikipedia, Enron, and UCI datasets. The results are depicted in Fig. 4. It is evident that the walk-based method, such as CAWN, requires a longer training time due to its inefficient temporal walk operations and has a substantial number of parameters. Conversely, simpler and memory-based methods such as DyRep and JODIE may possess fewer parameters and exhibit faster training times; however, they exhibit a

significant performance gap compared to the best-performing methods. In contrast, TGFormer outperforms the transformer-based method DyGFormer by achieving superior performance with a smaller size of trainable parameters and a moderate training time per epoch.

5.8. Ablation studies

We further validate the effectiveness of modules in TGFormer through ablation studies, including frequency encoding (FE), auto-correlation mechanics (ACoM), and adaptive readout (AR). We refer to TGFormer without these three modules as w/o FE, w/o ACoM, and w/o AR respectively, and to TGFormer without all as w/o ALL. In detail, Specifically, in w/o FE, the encoding alignment module only takes node/link encoding and time encoding as input. In w/o ACoM, we replace the proposed ACoM with the traditional attention mechanics [31]. In w/o AR, we replace the proposed adaptive readout with the traditional mean function [25]. We report the performance of the different modules on the six datasets in Fig. 5. We find that TGFormer typically performs best when all components are used, and the results are worse when any component is removed or changed. We can get the following conclusions: (1) removing the node interaction frequency encoding scheme significantly affects performance, as it directly captures the relationships between nodes and their interactions. (2) Although ACoM plays different roles on different datasets, on large datasets or datasets with significant periodicity, ACoM even plays a similar efficiency as FE. This suggests that ACoM can benefit from

Table 9TGFormer performance under different choices of hyper-parameter c in the auto-correlation mechanism.

Dataset	Wikipedia		Reddit		LastFM		Enron		UCI		CollegeMsg	
Metric	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC
$c = 1$	99.61	99.58	99.87	99.83	96.26	96.55	97.17	97.24	98.93	98.35	99.23	98.95
$c = 2$	99.97	99.97	99.88	99.84	96.28	96.54	96.62	96.68	99.02	98.65	99.29	99.09
$c = 3$	99.86	99.83	99.89	99.86	96.34	96.64	97.85	97.94	99.22	98.79	99.18	98.75
$c = 4$	99.58	99.57	99.87	99.82	96.44	95.70	97.56	97.72	99.24	98.82	99.28	98.94
$c = 5$	99.93	99.93	99.88	99.85	96.63	96.50	97.73	97.84	99.27	98.92	99.30	98.97

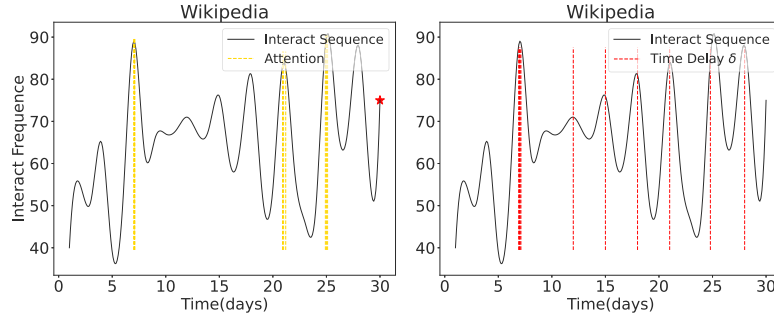


Fig. 7. Visualization of learned periodic dependencies. For clearness, we select the top-9 time delay sizes $\{\delta_1, \dots, \delta_9\}$ of Auto-Correlation and mark them in raw series (red lines). For attention, top-9 similar points with respect to the last time step (red stars) are also marked by yellow lines. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

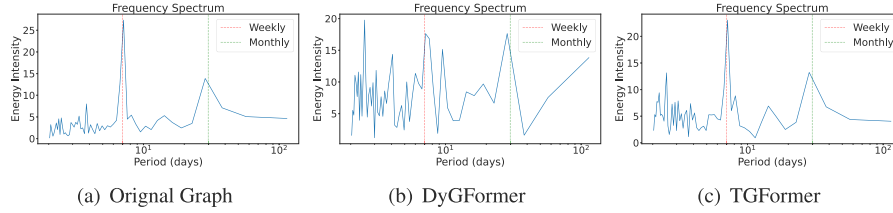


Fig. 8. The performance of different methods Original Graph(left), DyGFormer (middle), and TGFormer (right) in capturing periodic temporal patterns.

longer historical information and capture the periodic dependencies of the dataset. (3) The adaptive readout function can still achieve additional improvement.

5.9. Parameter sensitivity analysis

As demonstrated in Table 9, we can confirm the model's robustness concerning the hyper-parameter c (Eq. (9)). To achieve an optimal balance between performance and computational efficiency, we set c to the range of 1 to 5. It was also observed that the effects of most of the data sets did not fluctuate significantly and were more stable, all being somewhat periodic.

5.10. Long-term temporal dependency

We validate the advantages of our transformer architecture in effectively and efficiently utilizing longer histories. Our experiments are conducted on the Reddit and LastFM datasets, chosen for their potential to benefit from long-term historical data. For baseline comparisons, we augment the sampling of neighbors or increase the number of causal anonymous walks (starting from 2) to empower them with access to extended histories. The experimental results are illustrated in Fig. 6, with the x -axis depicted on a logarithmic scale with a base of 2. It is noteworthy that some baseline results are incomplete due to out-of-memory errors encountered at longer historical lengths. For instance, CAWN reaches an out-of-memory state when the historical length extends to 256. From Fig. 6, we deduce the following:

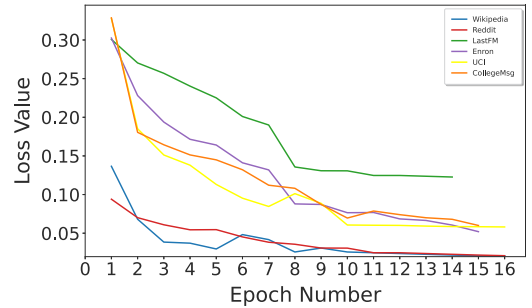


Fig. 9. The performance of TGFormer loss value on various datasets across epoch numbers.

(1) a majority of the baselines exhibit deteriorated performance with increasing historical lengths, indicative of their limitations in capturing long-term temporal dependencies; (2) the baselines generally incur substantial computational costs when processing extended histories. While memory network-based methods (e.g., TGN) manage to handle longer histories with reasonable computational costs, they do not significantly benefit from these extended histories due to issues such as vanishing or exploding gradients; (3) TGFormer consistently demonstrates performance gains with longer historical sequences, underscoring the superiority of the transformer architecture in effectively utilizing extended historical records.

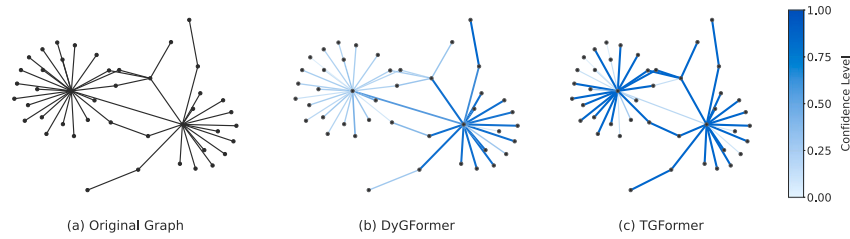


Fig. 10. Visualization of DyGFormer and TGFormer on link prediction performance.

5.11. Auto-correlation vs. attention

We also verified the advantage of the ACoM in capturing periodic temporal dependencies. We conducted experiments on Wikipedia to determine the model's effectiveness in capturing periodicity by measuring the interaction frequency of one of the nodes. For clarity, we visualized the learned periodic dependency in Fig. 7, where the x -axis represents the node interaction time divided by days, and the y -axis denotes the node interaction frequency during this period. For comparison with attention mechanisms, the top -9 time delay sizes $\{\delta_1, \dots, \delta_9\}$ of ACoM are highlighted within the raw series using red lines. In contrast, the top-9 most similar data points relative to the final time step, as determined by attention mechanisms, are denoted by yellow lines with red stars. As illustrated, while attention mechanisms capture node interaction intent, they fail to capture interaction periodicity effectively. This demonstrates that our model can discover relevant information more comprehensively and accurately. To more intuitively demonstrate TGFormer's ability to capture periodicity, we utilize a dataset generated by the Dynamic Stochastic Block Model (DSBM). This dataset consists of 1000 nodes, 24,430 edges, and spans one year with timestamps as the temporal granularity, exhibiting both weekly and monthly periodicity. Specifically, we provide the first 70% of the data to DyGFormer (attention mechanism) and TGFormer (auto-correlation mechanism) to generate the remaining 30%. By applying Fourier transform analysis to detect periodicity in the generated data, we validate TGFormer's capability to capture the inherent periodicity of time-evolving data. Our results, presented in Fig. 8, illustrate this capability: (a) represents the original generated graph data, which exhibits distinct weekly and monthly periodic patterns; (b) shows the graph data generated by DyGFormer; and (c) presents the graph data generated by TGFormer. These findings suggest that transforming data from the time domain to the frequency domain allows ACoM to more effectively identify key periodic patterns and correlations that may be difficult to detect in the time domain. In contrast, directly analyzing the time domain may fail to fully capture signal periodicity, particularly when the data contains noise or multiple overlapping frequency components. The frequency domain representation enables a clearer separation of different signal frequencies, thereby enhancing the identification of relevant patterns, especially for signals exhibiting periodic behavior.

5.12. Case study for link prediction

To illustrate the superiority of TGFormer more intuitively, we visualize the loss optimization process, as shown in Fig. 9. TGFormer's loss converges within 20 epochs across all datasets. By analyzing the convergence trends, we observe that datasets with a larger number of nodes exhibit faster convergence and lower corresponding loss values. This suggests that a greater number of node samples in each training iteration enhances the model's learning capability. Furthermore, we visualize TGFormer's advantages in link prediction, as shown in Fig. 10, where (a) represents the original graph, (b) shows the prediction results of DyGFormer, and (c) presents the prediction results of TGFormer. The

darkness of each edge indicates the probability assigned by the model to the existence of that edge. The results demonstrate that TGFormer outperforms DyGFormer by achieving more accurate link predictions.

6. Conclusion

In this paper, we introduce TGFormer, a novel approach that re-defines temporal graph learning by framing it within the context of time series analysis. Our model pioneers an auto-correlation mechanism that surpasses conventional attention-based methods, enabling the effective capture of long-term temporal dependencies and periodic patterns. This innovation not only enhances computational efficiency but also optimizes information utilization, making TGFormer well-suited for complex temporal graph tasks. Comprehensive experiments on diverse real-world datasets validate the model's effectiveness and efficiency, demonstrating its superior performance in temporal representation learning.

Despite its promising results, TGFormer has certain limitations. Notably, its reliance on one-hop neighbors restricts its ability to capture high-order relationships, which may hinder performance in scenarios requiring deeper relational modeling. Furthermore, its computational scalability remains a challenge, particularly for extremely large datasets. To address these limitations, future research will explore the integration of multi-hop neighbor aggregation to enrich the relational context and enhance model expressiveness. Additionally, we aim to investigate dynamic and scalable architectures capable of supporting large-scale data parallelism, a critical requirement for industrial applications that demand high efficiency and adaptability.

CRedit authorship contribution statement

Hongjiang Chen: Visualization, Methodology, Data curation, Formal analysis, Conceptualization, Writing – review & editing, Investigation, Writing – original draft, Project administration, Funding acquisition. **Pengfei Jiao:** Supervision, Project administration, Validation, Software, Writing – review & editing, Conceptualization, Resources, Methodology. **Ming Du:** Methodology. **Xuan Guo:** Project administration, Methodology, Writing – review & editing. **Zhidong Zhao:** Validation, Supervision, Writing – review & editing, Software. **Di Jin:** Validation, Supervision, Resources, Software. **Xiao Liu:** Formal analysis, Methodology, Writing – review & editing, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LDT23F01012F01, in part by the National Natural Science Foundation of China under Grant 62372146 and 62202304.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.patcog.2025.112053>.

Data availability

Data will be made available on request.

References

- [1] P. Jiao, H. Chen, X. Guo, Z. Zhao, D. He, D. Jin, A survey on temporal interaction graph representation learning: Progress, challenges, and opportunities, in: *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025.
- [2] Z. Wang, Y. Sun, X. Zhang, B. Xu, Z. Yang, H. Lin, Continual learning with high-order experience replay for dynamic network embedding, *Pattern Recognit.* 159 (2025) 111093.
- [3] W. Weng, J. Fan, H. Wu, Y. Hu, H. Tian, F. Zhu, J. Wu, A decomposition dynamic graph convolutional recurrent network for traffic forecasting, *Pattern Recognit.* 142 (2023) 109670.
- [4] S. Sun, X. Pan, S. Qi, J. Gao, Knowledge enhanced prompt learning framework for financial news recommendation, *Pattern Recognit.* (2025) 111461.
- [5] M. Li, Y. Gu, Y. Wang, Y. Fang, L. Bai, X. Zhuang, P. Lió, When hypergraph meets heterophily: New benchmark datasets and baseline, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 39, (17) 2025, pp. 18377–18384.
- [6] K.N. Kumar, D. Roy, T.A. Suman, C.K. Mohan, TSANet: Forecasting traffic congestion patterns from aerial videos using graphs and transformers, *Pattern Recognit.* 155 (2024) 110721.
- [7] W. Zhang, X. Lai, J. Wang, Social link inference via multiview matching network from spatiotemporal trajectories, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (4) (2020) 1720–1731.
- [8] P. Jiao, H. Chen, H. Tang, Q. Bao, L. Zhang, Z. Zhao, H. Wu, Contrastive representation learning on dynamic networks, *Neural Netw.* 174 (2024) 106240.
- [9] H. Chen, P. Jiao, H. Tang, H. Wu, Temporal graph representation learning with adaptive augmentation contrastive, in: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2023, pp. 683–699.
- [10] M. Qin, D.-Y. Yeung, Temporal link prediction: A unified framework, taxonomy, and review, *ACM Comput. Surv.* 56 (4) (2023) 1–40.
- [11] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, M. Bronstein, Temporal graph networks for deep learning on dynamic graphs, in: *ICML 2020 Workshop on Graph Representation Learning*, 2020.
- [12] Y. Wang, Y.-Y. Chang, Y. Liu, J. Leskovec, P. Li, Inductive representation learning in temporal networks via causal anonymous walks, in: *International Conference on Learning Representations*, 2021.
- [13] A. Souza, D. Mesquita, S. Kaski, V. Garg, Provably expressive temporal graph networks, *Adv. Neural Inf. Process. Syst.* 35 (2022) 32257–32269.
- [14] A. Casteigts, P. Flocchini, W. Quattrociocchi, N. Santoro, Time-varying graphs and dynamic networks, *Int. J. Parallel Emergent Distrib. Syst.* 27 (5) (2012) 387–408.
- [15] M. Li, A. Micheli, Y.G. Wang, S. Pan, P. Lió, G.S. Gnecco, M. Sanguineti, Guest editorial: Deep neural networks for graphs: Theory, models, algorithms, and applications, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (4) (2024) 4367–4372.
- [16] L. Yu, L. Sun, B. Du, W. Lv, Towards better dynamic graph learning: New architecture and unified library, *Adv. Neural Inf. Process. Syst.* 36 (2023) 67686–67700.
- [17] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, *Adv. Neural Inf. Process. Syst.* 34 (2021) 22419–22430.
- [18] T. Zhao, L. Fang, X. Ma, X. Li, C. Zhang, Tformer: A time–frequency domain bidirectional sequence-level attention based transformer for interpretable long-term sequence forecasting, *Pattern Recognit.* 158 (2025) 110994.
- [19] T. Dai, B. Wu, P. Liu, N. Li, X. Yuerong, S.-T. Xia, Z. Zhu, DDN: Dual-domain dynamic normalization for non-stationary time series forecasting, *Adv. Neural Inf. Process. Syst.* 37 (2024) 108490–108517.
- [20] D.R. Cox, *The Theory of Stochastic Processes*, Routledge, 2017.
- [21] P. Duhamel, M. Vetterli, Fast Fourier transforms: a tutorial review and a state of the art, *Signal Process.* 19 (4) (1990) 259–299.
- [22] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: *International Conference on Machine Learning*, PMLR, 2022, pp. 27268–27286.
- [23] Q. Wu, W. Zhao, C. Yang, H. Zhang, F. Nie, H. Jiang, Y. Bian, J. Yan, Sgformer: Simplifying and empowering transformers for large-graph representations, *Adv. Neural Inf. Process. Syst.* 36 (2023) 64753–64773.
- [24] Q. Wu, C. Yang, W. Zhao, Y. He, D. Wipf, J. Yan, DIFformer: Scalable (graph) transformers induced by energy constrained diffusion, in: *The Eleventh International Conference on Learning Representations*, 2023.
- [25] W. Cong, S. Zhang, J. Kang, B. Yuan, H. Wu, X. Zhou, H. Tong, M. Mahdavi, Do we really need complicated model architectures for temporal networks? in: *International Conference on Learning Representations*, 2023.
- [26] Y. Wu, Y. Fang, L. Liao, On the feasibility of simple transformer for dynamic graph modeling, in: *Proceedings of the ACM Web Conference 2024*, 2024, pp. 870–880.
- [27] S. Kumar, X. Zhang, J. Leskovec, Predicting dynamic embedding trajectory in temporal interaction networks, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 1269–1278.
- [28] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, K. Achan, Inductive representation learning on temporal graphs, in: *International Conference on Learning Representations*, 2020.
- [29] L. Luo, G. Haffari, S. Pan, Graph sequential neural ode process for link prediction on dynamic and sparse graphs, in: *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, 2023, pp. 778–786.
- [30] M. Jin, Y.-F. Li, S. Pan, Neural temporal walks: Motif-aware representation learning on continuous-time dynamic graphs, *Adv. Neural Inf. Process. Syst.* 35 (2022) 19874–19886.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [32] J. Chen, K. Gao, G. Li, K. He, NAGphormer: A tokenized graph transformer for node classification in large graphs, in: *The Eleventh International Conference on Learning Representations*, 2023.
- [33] H. Shirzad, A. Velingker, B. Venkatachalam, D.J. Sutherland, A.K. Sinop, Exphormer: Sparse transformers for graphs, in: *International Conference on Machine Learning*, PMLR, 2023, pp. 31613–31632.
- [34] Q. Wu, W. Zhao, Z. Li, D. Wipf, J. Yan, Nodeformer: A scalable graph structure learning transformer for node classification, in: *Advances in Neural Information Processing Systems*, 2022.
- [35] F. Poursafaei, S. Huang, K. Pelrine, R. Rabbany, Towards better evaluation for dynamic link prediction, *Adv. Neural Inf. Process. Syst.* 35 (2022) 32928–32941.
- [36] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, T. Liu, On layer normalization in the transformer architecture, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 10524–10533.
- [37] S. Narang, H.W. Chung, Y. Tay, L. Fedus, T. Févry, M. Matena, K. Malkan, N. Fiedel, N. Shazeer, Z. Lan, et al., Do transformer modifications transfer across implementations and applications? in: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 5758–5773.
- [38] R. Trivedi, M. Farajtabar, P. Biswal, H. Zha, Dyrep: Learning representations over dynamic graphs, in: *International Conference on Learning Representations*, 2019.
- [39] L. Wang, X. Chang, S. Li, Y. Chu, H. Li, W. Zhang, X. He, L. Song, J. Zhou, H. Yang, TcL: Transformer-based dynamic graph modelling via contrastive learning, 2021, arXiv preprint [arXiv:2105.07944](https://arxiv.org/abs/2105.07944).
- [40] Y. Tian, Y. Qi, F. Guo, Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction, in: *The Twelfth International Conference on Learning Representations*, 2024.

Hongjiang Chen received the B.E. degree in computer science from Jiangxi University of Science and Technology, Ganzhou, China, in 2022. He is currently pursuing a Ph.D. degree with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His research interests include deep learning and dynamic network learning.

Pengfei Jiao received the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2018. From 2018 to 2021, he was a lecturer with the Center of Biosafety Research and Strategy of Tianjin University. He is currently a Professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His current research interests include complex network analysis and its applications.

Ming Du received the Bachelor degree from Guilin University of Electronic Technology in 2021. He is currently pursuing a master's degree at the School of Cyberspace, Hangzhou Dianzi University. His current research interests include network embedding and graph generation.

Xuan Guo is pursuing a doctoral degree at the College of Intelligence and Computing, Tianjin University, P. R. China. His current research interests include complex network analysis, role discovery, network representation learning and percolation model.

Zhidong Zhao received the B.S. and M.S. degrees in mechanical engineering from the Nanjing University of Science and Technology, China, in 1998 and 2001, respectively, and the Ph.D. degree in biomedical engineering from Zhejiang University, China, in 2004. He is currently a Full Professor at Hangzhou Dianzi University, China. His research interests include biomedical signal processing, wireless sensor networks, biometrics, and machine learning.

Di Jin received the Ph.D. degree in computer science from Jilin University, Changchun, China, in 2012. He was a Post-Doctoral Research Fellow with the School of Design, Engineering, and Computing, Bournemouth University, Poole, U.K., from 2013 to 2014. He is currently an Associate Professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. He has authored or coauthored more than 50 papers in international journals and conferences in the areas of community detection, social network analysis, and machine learning. Dr. Jin serves as a Senior Program Committee Member for AAAI Conference on Artificial Intelligence.

Xiao Liu received the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2018. From 2018 to 2022, she was a postdoctor at School of Biomedical Engineering, Shanghai Jiao Tong University, Shanghai, China. She is currently an Assistant Researcher in Shanghai Cancer Institute, Shanghai, China. Her research interests include complex network analysis and application of deep learning to single-cell multi-omics data.